



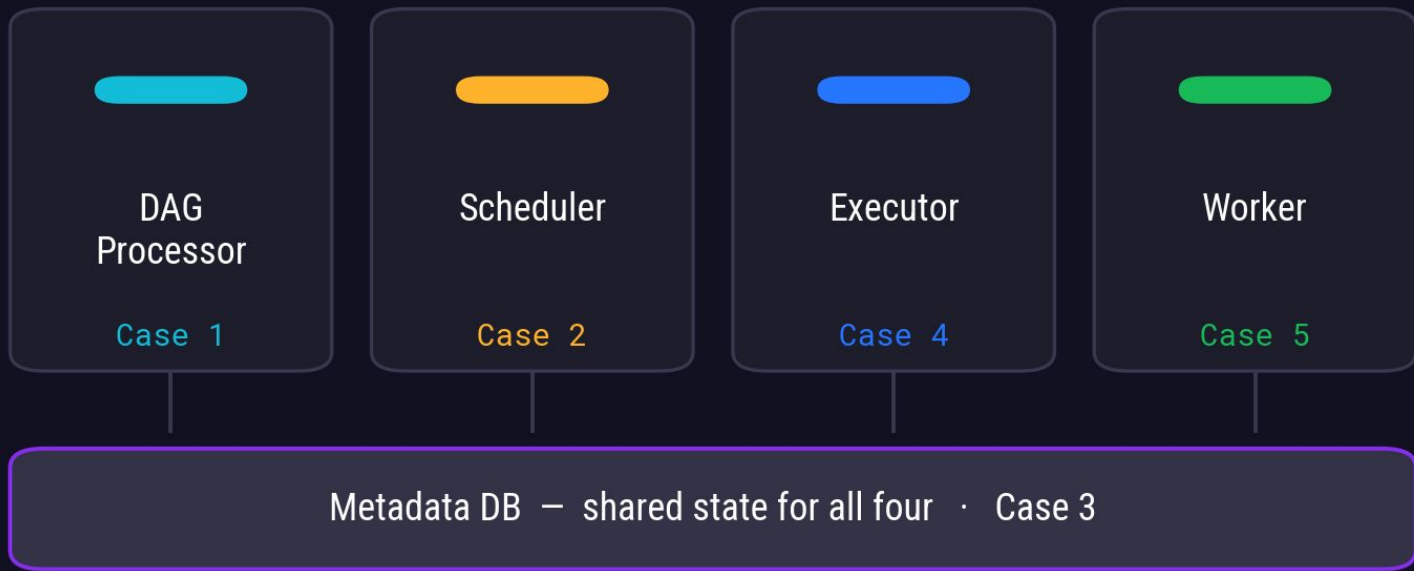
Debugging the Undebuggable

Five real incidents, and the signals that finally explained them.



Pankaj Singh Senior Software Engineer, Astronomer • Apache Airflow Committer

Why this is hard to debug.



The symptom rarely lives where the root cause does.

CASE 1

DAGs stopped importing after an Airflow upgrade.

Symptom — Every DAG fails to import, right after a routine upgrade.

First guess — “Our DAG code broke.”

What the logs actually said.



actual log output

```
dags/examples/dbt_analytics_mart.py:
```

```
mashumaro.exceptions.UnserializableField: Field "schema" of type  
Optional[str] in JSONObjectSchema is not serializable
```

```
dags/examples/dbt_ecommerce_core.py:
```

```
mashumaro.exceptions.UnserializableField: Field "schema" of type  
Optional[str] in JSONObjectSchema is not serializable
```

+ 2 more DAG files (dbt_staging.py, dbt_supply_chain.py) – identical error

What was really happening.



this surfaces as:

✗
Looks like: a DAG bug

✓
Actually: a version mismatch

CAUSE

A Python bump broke a pinned dependency underneath.

FIX

Pin back, then upgrade the dependency later.

A dependency pin plus a language bump can look like your own bug.

CASE 2

A successful task was marked as failed.

Symptom — A task succeeds on retry, then flips to failed with no new run.

First guess — “Flaky retry logic.”

What the logs actually said.



actual log output

```
[2026-03-12T01:55:19.149+0000] WARNING - Skipping stale failure  
callback for <TaskInstance:  
reproduce_zombie_race_condition.kill_and_retry  
manual__2026-03-12T01:50:36.556912+00:00 [success]>: callback  
try_number=1, current try_number=2
```

What was really happening.

**CAUSE**

A stale zombie callback lands late and overwrites the real result.

FIX

Skip callbacks that belong to an older try number.

A task can be marked failed by a mistake that already happened.

CASE 3

All DAGs slowed down overnight.

Symptom — Every DAG gets slower overnight, for reasons no one deployed.

First guess — “Must be yesterday’s deploy.”

What the logs actually said.

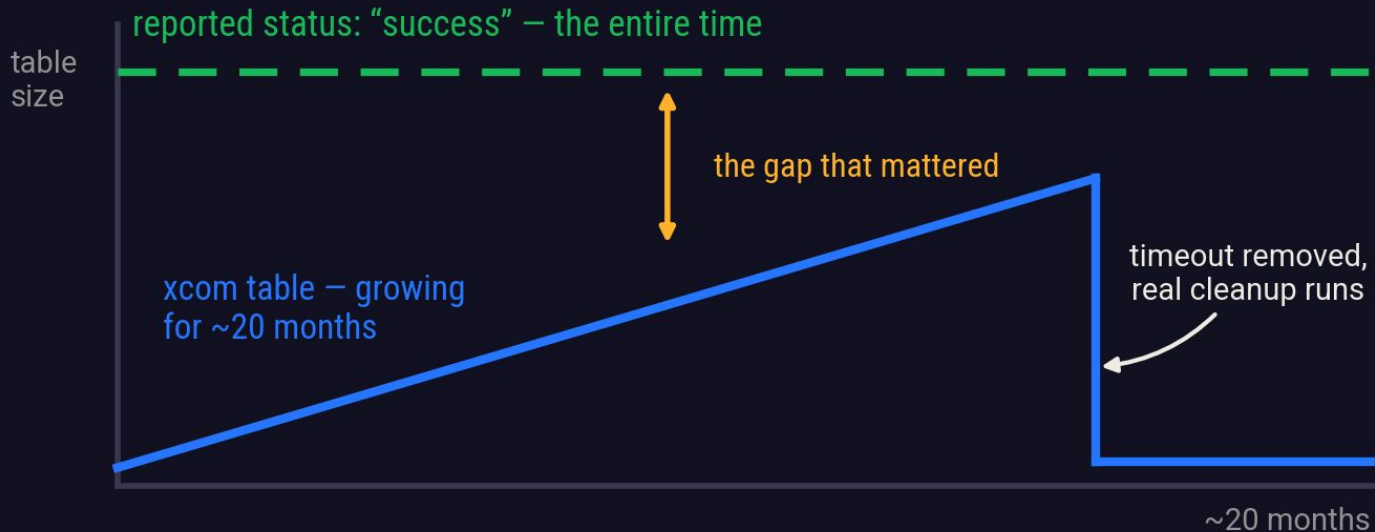


actual log output

```
WARNING - Encountered error when attempting to clean table 'xcom'.
```

```
psycopg2.errors.QueryCanceled: canceling statement due to statement  
timeout
```

What was really happening.

**CAUSE**

It caught the error and called it success anyway, for months.

FIX

Remove the timeout, then clear the backlog.

"Success" in the logs doesn't mean the work actually happened.

CASE 4

Tasks stuck in queue, pods not found.

Symptom — Tasks sit in the queue. “Pod not found” warnings everywhere.

First guess — “Kubernetes is flaky.”

What the logs actually said.



actual log output

```
WARNING - Cannot find pod for ti <TaskInstance:  
review_pack.build_report._concatenate_reports  
manual__2026-06-05T11:42:54.693597+00:00 [queued]>
```

```
INFO - 512 tasks up for execution:
```

What was really happening.

**CAUSE**

Two settings drifted far apart — 512 queued vs. only 16 pods.

FIX

Bring the two values back into line.

Two settings, each fine alone, can spin into a loop once they drift.

CASE 5

A task hung for hours with no error.

Symptom — A task hangs for hours, even though the work already finished.

First guess — “A new version regression.”

What the logs actually said.



actual log output

```
[16:55:45 UTC] ERROR - The recorded hostname 100.64.12.141 does not  
match this instance's hostname 100.64.9.18
```

```
[16:55:45 UTC] WARNING - Heartbeat failed with Exception: Hostname  
of job runner does not match
```

(repeats at 16:55:50 and 16:55:55 UTC, then two minutes of silence)

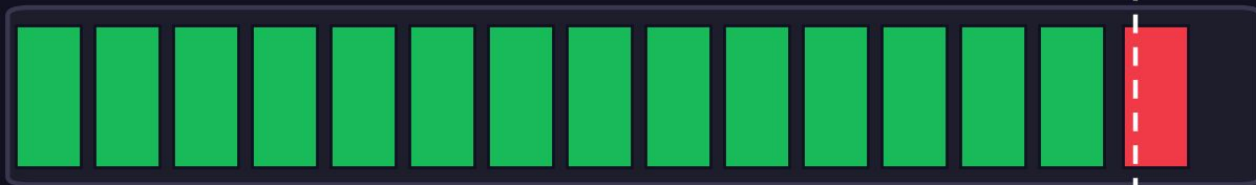
```
[16:58:11 UTC] ERROR - Job 2194108 was killed before it finished  
(likely due to running out of memory)
```

What was really happening.

WORKER MEMORY CAPACITY — 8.6GB

90% used — stable at 14

OOM



+1 task

14 tasks already fill ~90% of the 8.6GB worker

measured on Airflow 2 — worker composition varies by version and image

CAUSE

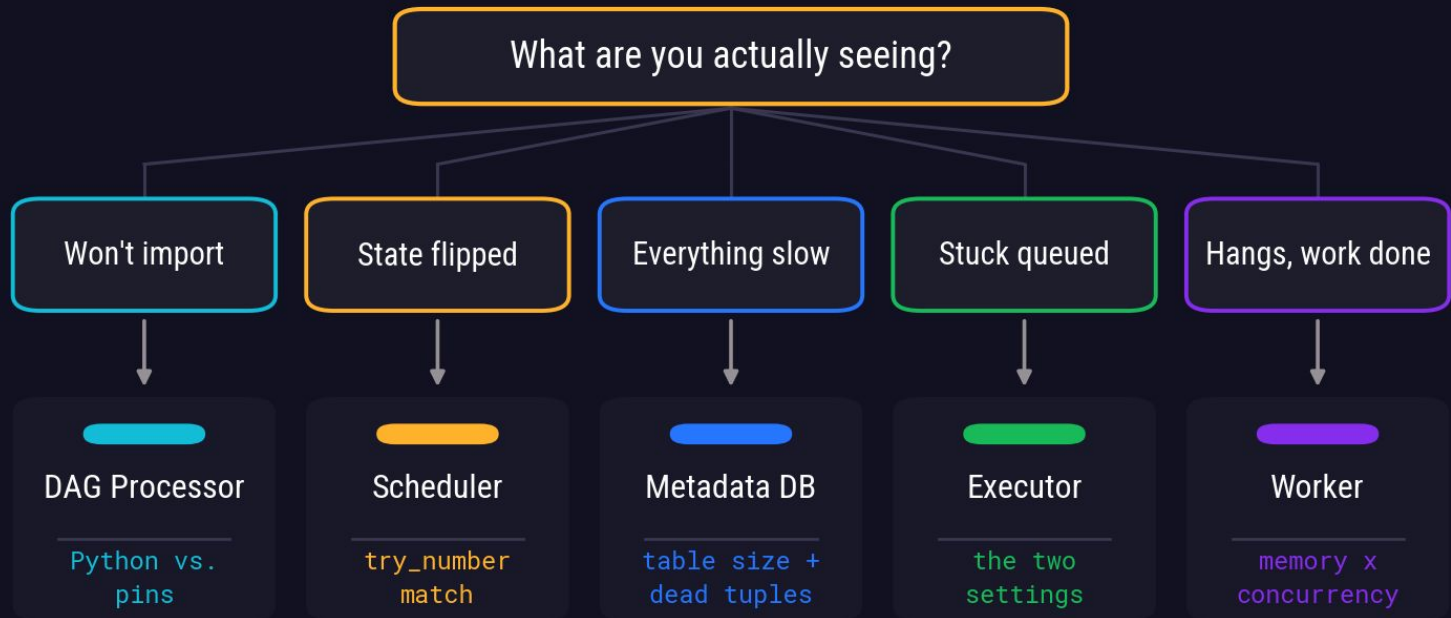
One extra unit of concurrency — 15 instead of 14 — tipped it over.

FIX

Drop concurrency by exactly one. No code change.

The root cause was never the version. It was simple arithmetic.

Which system is it?



Match the symptom to the system before you touch anything.

Where to look before you start guessing.

DAG fails to import after an upgrade

- › Runtime's Python version vs. your dependency pins
- › Installs clean but breaks at import = a transitive pin
- › Same error in every DAG = shared dep, not your code

Task state flips after it finished

- › Callback try_number vs. the task's current try_number
- › Zombie detection timing vs. when the retry succeeded
- › Audit log and state disagree = the state is wrong

Every DAG slows down at once

- › Rows the cleanup actually deleted, not its exit code
- › Metadata table size and dead tuples — xcom first
- › statement_timeout silently killing long deletes

Tasks stuck queued, "pod not found"

- › max_tis_per_query vs. worker_pods_creation_batch_size
- › "N tasks up for execution" vs. pods actually created
- › Kube API throttling and 429s during trigger bursts

Task hangs long after the work finished

- › Per-task memory × concurrency vs. worker memory
- › OOM kills in pod events, not just the task log
- › Concurrency differences between environments

Key takeaways.

Three habits that would have shortened every one of these.

1

Distrust the first guess

All five opened with a confident, reasonable, wrong hypothesis. In four of them the log itself pointed at the wrong component.

2

“Success” can lie

A caught exception reported success for over a year. A finished job looked hung. Task state is not the same as truth.

3

Check the boring numbers

Version pins, row counts, two settings side by side, memory math. Not one of these needed an exotic tool.

Thank you.

Questions?



Scan for LinkedIn

ASTRONJMER